MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova Faculty of Computers, Informatics and Microelectronics Department of Software and Automation Engineering

Postoronca Dumitru FAF-233

Report

Laboratory work n.6

on Embedded Systems

Checked by:
A. Martiniuc, *university assistant*
DISA, FCIM, UTM

Chișinău – 2026

1 Purpose of laboratory work

The purpose of this laboratory work is to implement an analog signal acquisition and conditioning system for a digital temperature sensor on the ESP32 microcontroller, using FreeRTOS for concurrent task execution and an OLED display plus STDIO for reporting.

1.1 Objectives of the laboratory work

Laboratory work has the following objectives that should be achieved:

- Implementing periodic sensor acquisition from a DHT11 digital temperature sensor at a configurable recurrence (2000 ms), exposing the raw value through an internal `sensor_read()` interface, with support for selecting a sensor from a catalogue of available sensors
- Implementing analog signal conditioning through a three-stage preprocessing pipeline: saturation (clamping to the sensor's physical range 0–50 °C), a median filter (window size 5) for salt-and-pepper noise removal, and an exponential moving average (EMA, $\alpha = 0.2$) for weighted smoothing
- Evaluating alert thresholds on the processed signal with hysteresis (ON $\geq$ 28 °C, OFF $\leq$ 24 °C) to generate a stable binary alert state that drives LED indicators
- Displaying a structured report at a lower recurrence (500 ms) on an OLED 128×64 display and via STDIO, showing the raw, median-filtered, and EMA-averaged temperature values, sensor validity, and alert state
- Structuring the firmware into three independent FreeRTOS tasks (Acquisition, Conditioning, Reporter) communicating through a binary semaphore and a mutex
- Organizing the codebase into three hardware-abstraction layers: MCAL, ECAL, and SRV

1.2 Problem definition

The application must be structured in three distinct FreeRTOS tasks:

1. **Task 1 – Sensor Acquisition (TaskAcquisition):** Reads raw temperature data from the currently selected sensor (DHT11) every 2000 ms. The active sensor is chosen at startup from a sensor catalogue. The raw value is stored in shared sensor data and a binary semaphore is given to signal the conditioning task.

2. **Task 2 – Signal Conditioning (TaskConditioning):** Blocked on the binary semaphore; wakes immediately when new sensor data arrives. Applies a three-stage preprocessing pipeline:

   (a) **Saturation** – clamps the raw value to the physical range of the sensor (0–50 °C for DHT11)

   (b) **Median filter** (window = 5) – removes salt-and-pepper outliers by sorting the last 5 saturated samples and selecting the median

   (c) **Exponential Moving Average** ($\alpha = 0.2$) – applies weighted smoothing: $y_n = \alpha \cdot x_n + (1 - \alpha) \cdot y_{n-1}$

   After filtering, the task evaluates alert thresholds with hysteresis (ON $\geq 28\,^{\circ}\text{C}$, OFF $\leq 24\,^{\circ}\text{C}$) and updates LED indicators: green LED when temperature is within normal range, red LED when the alert is active.

3. **Task 3 – Display and Reporting (TaskReporter):** Runs every 500 ms using `vTaskDelayUntil` for drift-corrected periodicity. Updates the OLED 128×64 display with all intermediate pipeline values (raw, median-filtered, EMA-averaged), sensor name, and alert state. Emits a serial plotter line in the `>name:value` format and every 5 seconds prints a full structured report including pipeline parameters and threshold configuration.

2 Domain Analysis

2.1 Application Context and Utilized Technologies

The primary objective of this laboratory work is to implement an **analog signal acquisition and conditioning system** for a digital temperature sensor using **FreeRTOS** on the ESP32 microcontroller. The application demonstrates a complete sensor pipeline: periodic raw data acquisition, multi-stage signal preprocessing (saturation, median filtering, weighted averaging), threshold-based alert detection, and structured reporting on both an OLED display and the serial interface.

The system uses the **DHT11** digital temperature and humidity sensor. The sensor communicates over a proprietary single-wire protocol on GPIO 4 and is read through the *Adafruit DHT* library. The DHT11 requires a minimum interval of approximately 2 seconds between consecutive reads, which determines the acquisition task period.

The signal conditioning stage implements three classical preprocessing techniques applied in sequence:

- **Saturation (clamping):** The raw sensor value is clamped to the physical measurement range of the DHT11 (0–50 °C). Values outside this range are replaced with the nearest boundary, preventing physically impossible readings from propagating through the pipeline.
- **Median filter (salt-and-pepper removal):** A sliding window of size 5 stores the most recent saturated samples. Each cycle, the window contents are sorted and the median value is selected. This non-linear filter is highly effective at removing isolated outlier spikes without distorting the underlying signal trend.
- **Exponential Moving Average (EMA):** A first-order IIR low-pass filter with coefficient $\alpha = 0.2$ is applied to the median-filtered output:

  $$y_n = \alpha \cdot x_n + (1 - \alpha) \cdot y_{n-1}$$

  This provides weighted smoothing where recent samples contribute more than older ones, reducing high-frequency noise while maintaining responsiveness.

After the full pipeline, alert thresholds with hysteresis (ON $\geq$ 28 °C, OFF $\leq$ 24 °C) are evaluated on the median-filtered value to produce a stable binary alert state.

Task synchronization is achieved through two FreeRTOS primitives:

- A **binary semaphore** (`xSemNewData`) given by TaskAcquisition after each read and taken by TaskConditioning, ensuring the conditioning logic runs exactly once per new sample
- A **mutex** (`xSensorMutex`) protecting all shared sensor data fields against concurrent access by the three tasks

2.2 Hardware and Software Components

Component	Description & Role
ESP32 DevKit	Dual-core microcontroller running FreeRTOS. Hosts all three application tasks and provides GPIO and I2C peripherals.
DHT11	Digital temperature and humidity sensor on GPIO 4. Single-wire protocol, 0–50 °C range, ±2 °C accuracy, minimum 2 s between reads.
Green LED	Connected to GPIO 25. Lights up when temperature is within the normal range (alert OFF).
Red LED	Connected to GPIO 26. Lights up when the temperature alert is active (temperature above 28 °C).
OLED 128×64 SSD1306	Connected on the I2C bus (SDA=GPIO 21, SCL=GPIO 22), address 0x3C. Primary display showing raw, filtered, and averaged values plus alert state at 500 ms intervals.
LCD 20×4 I2C	Connected on the same I2C bus, address 0x27. Backup display showing averaged temperature and alert state.
220 Ω **Resistors**	Current-limiting resistors for each LED.
10 kΩ **Pull-up**	Pull-up resistor on the DHT11 data line to 3.3 V for reliable communication.
Serial-to-USB Interface	UART at 115200 baud for serial plotter data (`>name:value` format) and periodic structured reports.
FreeRTOS	Built into the ESP32 Arduino core. Provides preemptive task scheduling, binary semaphore, and mutex.
PlatformIO	Build system and IDE integration used for compilation, upload, and serial monitoring.
Serial Plotter (VS Code)	VS Code extension for real-time plotting of data received via the serial port. Parses lines starting with `>` in `name:value` format and renders interactive time-series graphs, allowing side-by-side comparison of raw, median-filtered, and EMA-smoothed traces.
Breadboard	Prototyping base for connecting the DHT11, LEDs, OLED, LCD, and the ESP32 without soldering.

2.3 System Architecture and Solution Justification

The system is organized into three hardware-abstraction layers following the MCAL / ECAL / SRV pattern:

1. **MCAL (Microcontroller Abstraction Layer):** Contains only `GpioDriver`, which wraps `pinMode`, `digitalWrite`, `digitalRead`, and `analogRead`. All direct register / Arduino hardware calls are isolated here.

2. **ECAL (Electronic Component Abstraction Layer):** Contains component drivers: `TempSensor` (DHT11 via Adafruit DHT), `LedDriver`, `OledDisplay` (Adafruit SSD1306), `LcdDisplay` (LiquidCrystal_I2C), and `SerialIO`. Each module depends only on MCAL or its own library and exposes a clean C API. All pin assignments are declared in the respective header files for easy reference during circuit assembly.

3. **SRV (Service Layer):** Contains `SensorData` (shared data store with mutex/semaphore, configurable pipeline parameters, and sensor catalogue), and the three FreeRTOS task implementations: `TaskAcquisition`, `TaskConditioning`, and `TaskReporter`.

The data flow proceeds as follows:

1. **TaskAcquisition** reads the DHT11 sensor every 2000 ms, stores the raw temperature and validity flag in `SensorData` (under mutex), and gives the `xSemNewData` binary semaphore.

2. **TaskConditioning** unblocks immediately on the semaphore, reads the latest raw temperature, and applies the three-stage pipeline: saturation → median filter → EMA. It stores the intermediate and final values in `SensorData`, evaluates alert thresholds with hysteresis, and drives the green/red LEDs accordingly.

3. **TaskReporter** runs every 500 ms via `vTaskDelayUntil`, reads all shared values under mutex, updates the OLED (raw, median, EMA, alert) and LCD (backup), emits a serial plotter line, and every 5 seconds prints a full structured report to STDIO.

2.4 Relevant Case Study

A real-world application of multi-stage analog signal conditioning is **environmental monitoring in greenhouse control systems**. Raw temperature readings from sensors placed in a greenhouse are subject to several types of noise: sudden spikes from direct sunlight on the sensor (salt-and-pepper noise), electrical interference from nearby irrigation pumps, and gradual drift. A saturation stage rejects physically impossible readings (e.g., $-40\,^{\circ}\mathrm{C}$ indoors), a median filter removes the isolated spikes without smoothing away real temperature changes, and a weighted moving average provides the stable baseline needed for climate control decisions. Alert thresholds with hysteresis then trigger ventilation or heating actuators without the rapid on/off cycling that would occur with a single-threshold comparator – a pipeline directly reflected in this laboratory work.

3 Design

3.1 Architectural sketch

Below is the architectural sketch of the components structured in the diagram that highlights the software and hardware components.

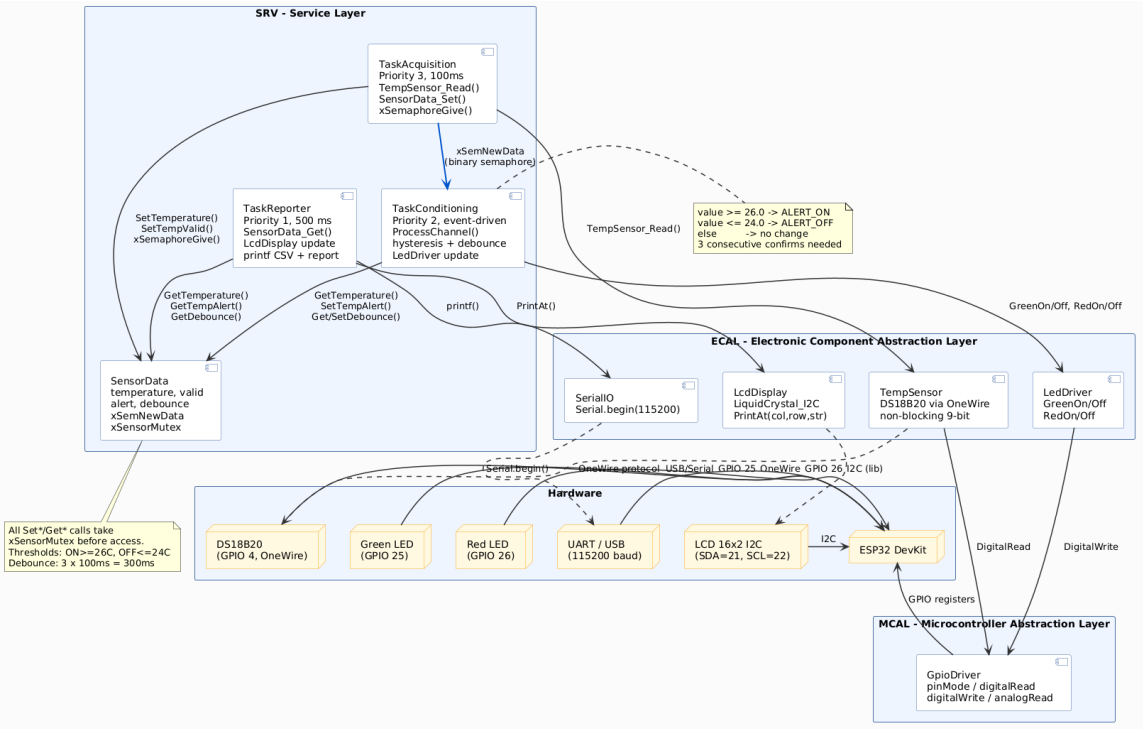

Figure 1: Architectural sketch of the project

This is the description of each of the elements of the diagram:

1. **ESP32 DevKit microcontroller** – central processing unit running FreeRTOS and executing all three application tasks preemptively
2. **DHT11 temperature sensor (GPIO 4)** – digital sensor communicating over a single-wire protocol; read by TaskAcquisition every 2000 ms
3. **Green LED (GPIO 25)** – output indicator; ON when temperature alert is OFF

4. **Red LED (GPIO 26)** – output indicator; ON when temperature alert is active
5. **OLED 128×64 SSD1306 (SDA=21, SCL=22, addr 0x3C)** – primary display updated by TaskReporter every 500 ms with raw, filtered, averaged values and alert state
6. **LCD 20×4 I2C (SDA=21, SCL=22, addr 0x27)** – backup display showing averaged temperature and alert state
7. **SensorData module** – shared data store protected by `xSensorMutex`; holds raw, filtered, and averaged temperature values, validity flag, alert state, sensor selection, and all configurable pipeline parameters
8. **TaskAcquisition** – FreeRTOS task (priority 3) that reads the sensor periodically and gives `xSemNewData`
9. **TaskConditioning** – FreeRTOS task (priority 2) blocked on `xSemNewData`; applies saturation, median filter, and EMA; evaluates alert thresholds; updates LEDs
10. **TaskReporter** – FreeRTOS task (priority 1) that updates OLED, LCD, and prints STDIO reports at 500 ms intervals
11. **GpioDriver (MCAL)** – wraps all `digitalRead`/`digitalWrite` calls; only layer that touches Arduino hardware APIs directly

3.2 Electrical sketch

The circuit diagram outlines the setup for this project, showing the connections between the ESP32, two LEDs (green, red), the DHT11 temperature sensor, the OLED and LCD I2C modules, and the computer for serial communication. Each LED is connected to its respective GPIO pin through a 220 Ω current-limiting resistor. The DHT11 data line (GPIO 4) requires a 10 kΩ pull-up resistor to 3.3 V for reliable communication. Both displays share the I2C bus (SDA=GPIO 21, SCL=GPIO 22) at different addresses (OLED at 0x3C, LCD at 0x27).

Figure **??** details the connections and components that make up the system. This visual documentation is crucial for replicating the setup or diagnosing issues, as it clearly shows how each component is integrated into the overall design.

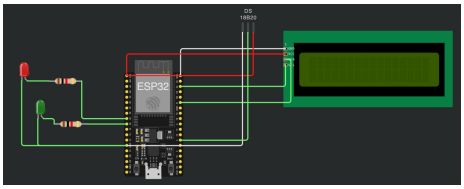

Figure 2: Circuit sketched on wokwi

3.3 Flow chart

The Figure **??** illustrates the FreeRTOS task interaction flow. TaskAcquisition runs every 2000 ms, reads the DHT11, updates shared data under the mutex, and gives the binary semaphore. TaskConditioning unblocks immediately, reads the shared raw temperature, applies the three-stage conditioning pipeline (saturation → median filter → EMA), stores intermediate results, evaluates alert thresholds, and updates the LEDs. TaskReporter runs every 500 ms independently and reads shared data to refresh the OLED, LCD, and serial output.

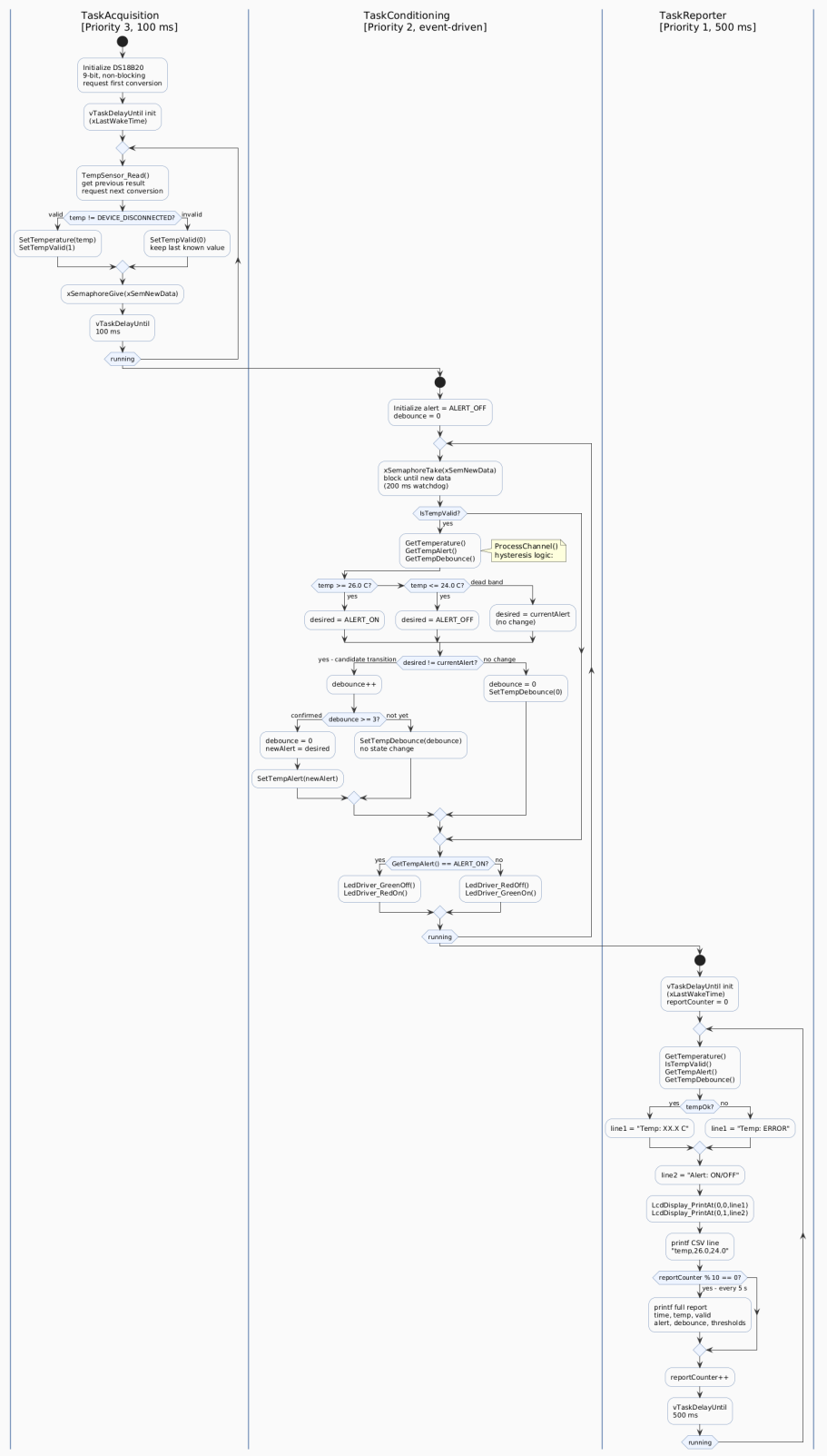

Figure 3: Flow chart of the algorithm

3.4 Code design

Code is developed following the principle of separating the interface from the implementation, by creating for each header file `.h` a respective `.cpp` file. The project uses **PlatformIO** with three library directories:

1. **MCAL layer** (`lib/MCAL/GpioDriver`) – hardware-specific code that wraps `Arduino.h` GPIO calls. All direct hardware access is encapsulated here.
2. **ECAL layer** (`lib/ECAL/`) – electronic component drivers: `TempSensor` (DHT11 via Adafruit DHT library), `LedDriver` (two LEDs), `OledDisplay` (Adafruit SSD1306 128×64), `LcdDisplay` (LiquidCrystal_I2C, backup), and `SerialIO` (UART initialization). Each driver header declares its connection pins for easy circuit reference.
3. **SRV layer** (`lib/SRV/`) – the `SensorData` shared-state module and the three FreeRTOS task implementations.

The `SensorData` module centralizes all shared data behind mutex-protected getters and setters, and owns the FreeRTOS synchronization primitives:

```
// Declared in SensorData.h, initialized in SensorData_Init()
extern SemaphoreHandle_t xSemNewData;   // binary semaphore: new
    data available
extern SemaphoreHandle_t xSensorMutex;  // mutex: protects all
    shared sensor fields
```

The signal conditioning logic is implemented as a three-stage pipeline in `TaskConditioning`:

```
/* 1. Saturation -- clamp to physical sensor range */
static float Saturate(float val)
{
    if (val < SATURATION_LOW)  return SATURATION_LOW;
    if (val > SATURATION_HIGH) return SATURATION_HIGH;
    return val;
}

/* 2. Median filter -- sort window, return middle element */
static float MedianFilter_Get(void)
{
    float sorted[MEDIAN_WINDOW];
    uint8_t n = medianFilled;
    for (uint8_t i = 0; i < n; i++) sorted[i] = medianBuf[i];
```

```
    // Insertion sort (n <= 5)
    for (uint8_t i = 1; i < n; i++) {
        float key = sorted[i];
        int j = i - 1;
        while (j >= 0 && sorted[j] > key) {
            sorted[j + 1] = sorted[j]; j--;
        }
        sorted[j + 1] = key;
    }
    return sorted[n / 2];
}

/* 3. Exponential Moving Average */
static float EMA_Update(float newVal)
{
    if (!emaInitialised) {
        emaValue = newVal;
        emaInitialised = 1;
    } else {
        emaValue = EMA_ALPHA * newVal
                 + (1.0f - EMA_ALPHA) * emaValue;
    }
    return emaValue;
}
```

TaskReporter uses `vTaskDelayUntil` for drift-corrected 500 ms periodicity and outputs data in the `>name:value` format compatible with the VS Code Serial Plotter extension, alongside a full structured report every 5 seconds:

```
// Serial Plotter line (every 500 ms)
Serial.print(">");
Serial.print("raw:"); Serial.print(raw, 2);
Serial.print(",median:"); Serial.print(filtered, 2);
Serial.print(",ema:"); Serial.print(averaged, 2);
Serial.print(",thresh_high:"); Serial.print(TEMP_THRESH_HIGH, 1);
Serial.print(",thresh_low:"); Serial.print(TEMP_THRESH_LOW, 1);
Serial.println();

// Full structured report (every 5 s)
if ((reportCounter++ % FULL_REPORT_EVERY) == 0) {
    printf("\n===== SENSOR REPORT (t=%lu ms) =====\n", ...);
    printf("Raw           : %.2f C\n", raw);
    printf("Saturated+Med : %.2f C  (median window=%d)\n",
           filtered, MEDIAN_WINDOW);
    printf("EMA averaged  : %.2f C  (alpha=%.2f)\n",
```

```
17              averaged, EMA_ALPHA);
18      printf("Alert          : %s  (ON>=%.1f OFF<=%.1f)\n", ...);
19      printf("====================================\n\n");
20  }
```

4 Results

For running the application the following commands were used for compiling, uploading the project and monitoring the port to observe the periodic reports.

```
# list connected boards and their ports
pio device list
# compile and upload to ESP32
pio run --target upload
# open serial monitor at 115200 baud
pio device monitor --baud 115200
```

Below is the output of the commands listed above

```
====================================
  Lab 6 - Analog Signal Acquisition
====================================
Sensor    : DHT11 (digital) GPIO4
Acq.period: 2000 ms
Pipeline  : Saturation -> Median(5) -> EMA(a=0.20)
Thresholds: ON >= 28.0C, OFF <= 24.0C
Display   : OLED 128x64 SSD1306 + LCD 20x4 + Serial
====================================

>raw:24.00,median:24.00,ema:24.00,thresh_high:28.0,thresh_low:24.0
>raw:24.00,median:24.00,ema:24.00,thresh_high:28.0,thresh_low:24.0
>raw:25.00,median:24.00,ema:24.00,thresh_high:28.0,thresh_low:24.0

===== SENSOR REPORT (t=5000 ms) =====
Active sensor : DHT11
Validity      : OK
Raw           : 25.00 C
Saturated+Med : 24.00 C  (median window=5)
EMA averaged  : 24.00 C  (alpha=0.20)
Alert         : OFF  (thresholds: ON>=28.0 OFF<=24.0)
====================================

>raw:29.00,median:25.00,ema:24.20,thresh_high:28.0,thresh_low:24.0
>raw:30.00,median:29.00,ema:25.16,thresh_high:28.0,thresh_low:24.0
```

```
>raw:30.00,median:30.00,ema:26.13,thresh_high:28.0,thresh_low:24.0

===== SENSOR REPORT (t=10000 ms) =====
Active sensor : DHT11
Validity      : OK
Raw           : 30.00 C
Saturated+Med : 30.00 C  (median window=5)
EMA averaged  : 27.10 C  (alpha=0.20)
Alert         : ON  (thresholds: ON>=28.0 OFF<=24.0)
======================================
```

The system behavior is as follows:

- At startup, the OLED shows `Lab6 SigAcquire` / `Starting...` for 1 second while all modules initialize.
- During normal operation (temperature below 28 °C), the **green LED** is on and the OLED displays all pipeline stages: raw temperature, median-filtered value, and EMA-averaged value, along with the sensor name and alert state.
- The serial plotter line in `>name:value` format allows real-time visualization in the VS Code Serial Plotter extension, where the raw, median, and EMA traces can be compared side by side against the threshold reference lines.
- When the temperature rises above 28 °C (as seen by the median-filtered value), the alert transitions to ON: the **red LED** turns on, the green LED turns off, and the OLED display updates to show `Alert:  ON`.
- The alert turns OFF only after the median-filtered temperature drops below 24 °C, preventing chattering in the 24–28 °C hysteresis band.
- The EMA trace on the serial plotter visibly lags behind the raw and median traces, demonstrating the smoothing effect of the weighted average with $\alpha = 0.2$.
- Every 500 ms, a serial plotter data line is emitted. Every 5 seconds, a full structured report is printed showing the active sensor, validity, all pipeline values with their parameters, and alert configuration.

Below is the visual representation of the circuit

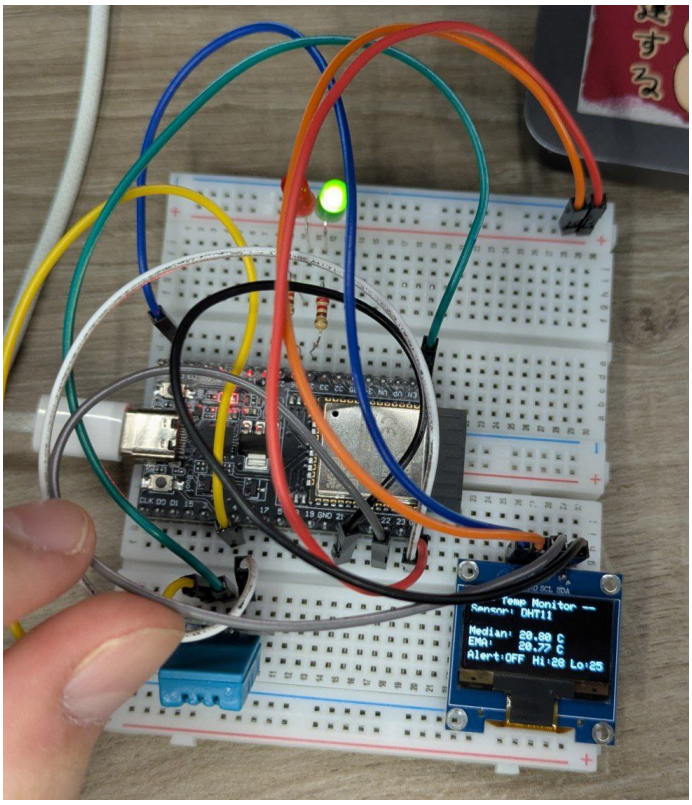

Figure 4: Circuit in working state

Figure **??** shows the circuit in its idle state with both LEDs off before acquisition begins.

This is how the plotting of values in time look:

Figure 5: Ploting the sensor values and filtered data

5 Conclusion

Completion of this laboratory work has provided practical understanding of analog signal acquisition and conditioning techniques for embedded sensor systems, and reinforced the use of FreeRTOS synchronization primitives for safe inter-task communication.

This sixth laboratory work covered the complete sensor pipeline from raw data acquisition through multi-stage signal conditioning to structured reporting. The key takeaways are:

1. Implementing **periodic sensor acquisition** with the DHT11 in FreeRTOS context – the sensor's minimum 2 second interval between reads defines the acquisition period, and the active sensor is selected from a configurable catalogue, allowing the system to be extended with additional sensor types without modifying the task logic.

2. Applying a **three-stage signal conditioning pipeline**:

   - **Saturation** clamps the raw value to the sensor's physical range (0–50 °C), rejecting impossible readings at the input boundary.
   - **Median filter** (window = 5) removes salt-and-pepper noise – isolated outlier spikes are eliminated without distorting the underlying temperature trend, as the median is inherently robust to up to $\lfloor n/2 \rfloor$ outliers in the window.
   - **Exponential Moving Average** ($\alpha = 0.2$) provides weighted smoothing where recent samples contribute 20% and the accumulated history contributes 80%, yielding a smooth output suitable for display while maintaining adequate responsiveness to real temperature changes.

3. Evaluating **alert thresholds with hysteresis** (ON $\geq$ 28 °C, OFF $\leq$ 24 °C) on the median-filtered value rather than the heavily smoothed EMA, ensuring that the alert responds promptly to actual temperature crossings while the dead band prevents chattering near the threshold.

4. Structuring the firmware into three distinct **FreeRTOS tasks** with well-defined roles and priorities: Acquisition (priority 3) runs most frequently and ensures fresh data, Conditioning (priority 2) is event-driven via semaphore and reacts immediately to new samples, and Reporter (priority 1) provides periodic human-readable output without interfering with the control loop.

5. Using a **mutex-protected shared data store** (`SensorData`) as the single source of truth for all sensor state – raw, filtered, and averaged values along with validity and alert state – preventing data races when multiple tasks read or write concurrently. All configurable pipeline parameters (acquisition period, saturation limits, median window, EMA coefficient, thresholds) are centralized in the `SensorData.h` header.

6. Organizing the project into **three hardware-abstraction layers** (MCAL, ECAL, SRV) with PlatformIO, keeping all hardware-specific code in the MCAL layer, declaring all pin assignments in ECAL driver headers for easy circuit reference, and making the service layer fully portable across different MCU platforms.

This laboratory work demonstrates how a pipeline of classical signal processing techniques – saturation, median filtering, and exponential moving averaging – combined with FreeRTOS synchronization primitives, transforms a noisy raw sensor stream into a clean, reliable signal suitable for both display and alert generation in an embedded system.

References

[1] Espressif Systems. (2023). *ESP32 Series Datasheet*. Retrieved from https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[2] Aosong Electronics. (2020). *DHT11 Humidity & Temperature Sensor Datasheet*. Retrieved from https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf

[3] Adafruit Industries. (2024). *DHT sensor library for Arduino*. Retrieved from https://github.com/adafruit/DHT-sensor-library

[4] Adafruit Industries. (2024). *Adafruit SSD1306 OLED Driver Library*. Retrieved from https://github.com/adafruit/Adafruit_SSD1306

[5] Real Time Engineers Ltd. (2024). *FreeRTOS API Reference*. Retrieved from https://www.freertos.org/a00106.html

[6] Barry, R. (2016). *Mastering the FreeRTOS Real Time Kernel – A Hands-On Tutorial Guide*. Real Time Engineers Ltd. Retrieved from https://www.freertos.org/Documentation/RTOS_book.html

[7] Barr, M. (2006). *Programming Embedded Systems: With C and GNU Development Tools*. O'Reilly Media, Inc.

[8] Smith, S. W. (1997). *The Scientist and Engineer's Guide to Digital Signal Processing*. California Technical Publishing. Retrieved from https://www.dspguide.com/

[9] Oppenheim, A. V., Willsky, A. S., & Nawab, S. H. (1996). *Signals and Systems* (2nd ed.). Prentice Hall.

A Source Code

A.1 Main Entry Point

```
1
2  #include "SerialIO.h"
3  #include "TempSensor.h"
4  #include "LedDriver.h"
5  #include "LcdDisplay.h"
6  #include "OledDisplay.h"
7  #include "SensorData.h"
8  #include "TaskAcquisition.h"
9  #include "TaskConditioning.h"
10 #include "TaskReporter.h"
11
12 void setup()
13 {
14     /*        Driver / ECAL initialisation        */
15     SerialIoInit();
16     TempSensor_Init();   // DS18B20 on GPIO4, 9-bit non-blocking
17     LedDriver_Init();
18     LcdDisplay_Init();   // LCD 16x2 I2C (0x27, SDA=21, SCL=22)
19     OledDisplay_Init();  // OLED 128x64 SSD1306 I2C (0x3C)
20
21     /*        Service layer initialisation        */
22     SensorData_Init();   // semaphore + mutex
23     SensorData_SetActiveSensor(SENSOR_DHT11);
24
25     printf("\n=====================================\n");
26     printf("  Lab 6 - Analog Signal Acquisition\n");
27     printf("=====================================\n");
28     printf("Sensor     : DHT11 (digital) GPIO%d\n", TEMP_SENSOR_PIN);
29     printf("Acq.period: %d ms\n", ACQUISITION_PERIOD_MS);
30     printf("Pipeline   : Saturation -> Median(%d) -> EMA(a=%.2f)\n",
31            MEDIAN_WINDOW, EMA_ALPHA);
32     printf("Thresholds: ON >= %.1fC, OFF <= %.1fC\n",
33            TEMP_THRESH_HIGH, TEMP_THRESH_LOW);
34     printf("Display    : OLED 128x64 SSD1306 + LCD 16x2 + Serial\n");
35     printf("=====================================\n\n");
36
37     LcdDisplay_PrintAt(0, 0, "Lab6 SigAcquire");
38     LcdDisplay_PrintAt(0, 1, "Starting...");
39
40     OledDisplay_Clear();
41     OledDisplay_SetTextSize(1);
```

```
42      OledDisplay_PrintAt(0, 0, "Lab6 SigAcquire");
43      OledDisplay_PrintAt(0, 16, "Starting...");
44      OledDisplay_Update();
45
46      vTaskDelay(pdMS_TO_TICKS(1000));
47
48      /*        FreeRTOS tasks (priorities: Acq=3, Cond=2, Reporter=1)
                */
49      xTaskCreate(TaskAcquisition_Task,  "Acquisition",  2048, NULL,
        3, NULL);
50      xTaskCreate(TaskConditioning_Task, "Conditioning", 2048, NULL,
        2, NULL);
51      xTaskCreate(TaskReporter_Task,     "Reporter",     4096, NULL,
        1, NULL);
52  }
53
54  void loop()
55  {
56      vTaskDelay(portMAX_DELAY);
57  }
```

Listing 1: main.cpp - Application Entry Point

A.2 MCAL Layer

```
1  #pragma once
2  #include <Arduino.h>
3
4  void    GpioDriver_PinMode(uint8_t pin, uint8_t mode);
5  void    GpioDriver_Write(uint8_t pin, uint8_t val);
6  uint8_t GpioDriver_Read(uint8_t pin);
7  void    GpioDriver_Toggle(uint8_t pin);
```

Listing 2: GpioDriver.h - GPIO Abstraction Header

```
1  #include "GpioDriver.h"
2
3  void GpioDriver_PinMode(uint8_t pin, uint8_t mode)
4  {
5      pinMode(pin, mode);
6  }
7
8  void GpioDriver_Write(uint8_t pin, uint8_t val)
9  {
10     digitalWrite(pin, val);
```

```
}

uint8_t GpioDriver_Read(uint8_t pin)
{
    return digitalRead(pin);
}

void GpioDriver_Toggle(uint8_t pin)
{
    digitalWrite(pin, !digitalRead(pin));
}
```

Listing 3: GpioDriver.cpp - GPIO Abstraction Implementation

A.3 ECAL Layer – Sensors and Peripherals

```
#pragma once

#define TEMP_SENSOR_PIN 4

void  TempSensor_Init(void);
float TempSensor_Read(void);
int   TempSensor_IsValid(void);
```

Listing 4: TempSensor.h - DHT11 Driver Header

```
#include "TempSensor.h"
#include <DHT.h>

static DHT dht(TEMP_SENSOR_PIN, DHT11);
static float lastTemp  = 20.0f;
static int   lastValid = 0;

void TempSensor_Init(void)
{
    dht.begin();
    delay(1000);  // DHT11 needs ~1s startup
}

float TempSensor_Read(void)
{
    float temp = dht.readTemperature();
    lastValid = !isnan(temp);
    if (lastValid) {
        lastTemp = temp;
```

```
    }
    return lastTemp;
}

int TempSensor_IsValid(void)
{
    return lastValid;
}
```

Listing 5: TempSensor.cpp - DHT11 Driver Implementation

```
#pragma once

/* --- Wiring --- */
#define LED_GREEN_PIN  25
#define LED_RED_PIN    26

void LedDriver_Init(void);
void LedDriver_GreenOn(void);
void LedDriver_GreenOff(void);
void LedDriver_RedOn(void);
void LedDriver_RedOff(void);
```

Listing 6: LedDriver.h - LED Driver Header

```
#include "LedDriver.h"
#include "GpioDriver.h"

void LedDriver_Init(void)
{
    GpioDriver_PinMode(LED_GREEN_PIN, OUTPUT);
    GpioDriver_PinMode(LED_RED_PIN,   OUTPUT);
    GpioDriver_Write(LED_GREEN_PIN, LOW);
    GpioDriver_Write(LED_RED_PIN,   LOW);
}

void LedDriver_GreenOn(void)  { GpioDriver_Write(LED_GREEN_PIN, HIGH); }
void LedDriver_GreenOff(void) { GpioDriver_Write(LED_GREEN_PIN, LOW); }
void LedDriver_RedOn(void)    { GpioDriver_Write(LED_RED_PIN,   HIGH); }
void LedDriver_RedOff(void)   { GpioDriver_Write(LED_RED_PIN,   LOW); }
```

Listing 7: LedDriver.cpp - LED Driver Implementation

```h
#pragma once
#include <stdint.h>

/* --- Wiring (I2C, shared bus with LCD) --- */
#define OLED_WIDTH      128
#define OLED_HEIGHT      64
#define OLED_I2C_ADDR  0x3C
#define OLED_SDA_PIN    21   /* ESP32 default I2C SDA */
#define OLED_SCL_PIN    22   /* ESP32 default I2C SCL */

void OledDisplay_Init(void);
void OledDisplay_Clear(void);
void OledDisplay_SetCursor(uint8_t x, uint8_t y);
void OledDisplay_SetTextSize(uint8_t size);
void OledDisplay_Print(const char *str);
void OledDisplay_PrintAt(uint8_t x, uint8_t y, const char *str);
void OledDisplay_Update(void);
```

Listing 8: OledDisplay.h - OLED SSD1306 Driver Header

```cpp
#include "OledDisplay.h"
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

static Adafruit_SSD1306 oled(OLED_WIDTH, OLED_HEIGHT, &Wire, -1);

void OledDisplay_Init(void)
{
    oled.begin(SSD1306_SWITCHCAPVCC, OLED_I2C_ADDR);
    oled.clearDisplay();
    oled.setTextColor(SSD1306_WHITE);
    oled.setTextSize(1);
    oled.display();
}

void OledDisplay_Clear(void)
{
    oled.clearDisplay();
}

void OledDisplay_SetCursor(uint8_t x, uint8_t y)
{
    oled.setCursor(x, y);
}

```

```
void OledDisplay_SetTextSize(uint8_t size)
{
    oled.setTextSize(size);
}

void OledDisplay_Print(const char *str)
{
    oled.print(str);
}

void OledDisplay_PrintAt(uint8_t x, uint8_t y, const char *str)
{
    oled.setCursor(x, y);
    oled.print(str);
}

void OledDisplay_Update(void)
{
    oled.display();
}
```

Listing 9: OledDisplay.cpp - OLED SSD1306 Driver Implementation

```
#pragma once
#include <stdint.h>

/* --- Wiring (I2C, shared bus with OLED) --- */
#define LCD_I2C_ADDR   0x27
#define LCD_COLS       20
#define LCD_ROWS        4
#define LCD_SDA_PIN    21   /* ESP32 default I2C SDA */
#define LCD_SCL_PIN    22   /* ESP32 default I2C SCL */

void LcdDisplay_Init(void);
void LcdDisplay_Clear(void);
void LcdDisplay_SetCursor(uint8_t col, uint8_t row);
void LcdDisplay_Print(const char *str);
void LcdDisplay_PrintAt(uint8_t col, uint8_t row, const char *str);
```

Listing 10: LcdDisplay.h - LCD I2C Driver Header

```
#include "LcdDisplay.h"
#include <LiquidCrystal_I2C.h>

static LiquidCrystal_I2C lcd(LCD_I2C_ADDR, LCD_COLS, LCD_ROWS);

```

```
void LcdDisplay_Init(void) {
  lcd.init();
  lcd.backlight();
  lcd.clear();
}

void LcdDisplay_Clear(void) { lcd.clear(); }

void LcdDisplay_SetCursor(uint8_t col, uint8_t row) { lcd.setCursor(
    col, row); }

void LcdDisplay_Print(const char *str) { lcd.print(str); }

void LcdDisplay_PrintAt(uint8_t col, uint8_t row, const char *str) {
  lcd.setCursor(col, row);
  lcd.print(str);
}
```

Listing 11: LcdDisplay.cpp - LCD I2C Driver Implementation

```
#pragma once

#define SERIAL_BAUD 115200

void SerialIoInit(void);
```

Listing 12: SerialIO.h - Serial Initialization Header

```
#include "SerialIO.h"
#include <Arduino.h>

void SerialIoInit(void)
{
    Serial.begin(SERIAL_BAUD);
    delay(2000);
}
```

Listing 13: SerialIO.cpp - Serial Initialization Implementation

A.4 SRV Layer – Shared Data

```
#pragma once
#include <freertos/FreeRTOS.h>
#include <freertos/semphr.h>
#include <stdint.h>
```

```
5
6  /*        Configurable parameters        */
7  #define ACQUISITION_PERIOD_MS 100  /* DHT11 requires >= 2s between
       reads   */
8  #define CONDITIONING_PERIOD_MS 100 /* can equal or be a multiple of
       above  */
9  #define REPORTER_PERIOD_MS 500     /* display / report cadence
             */
10 #define FULL_REPORT_EVERY 10       /* 10 x 500ms = 5 s full report
             */
11
12 /* Saturation limits ( C )     DHT11 range: 0..50  C  */
13 #define SATURATION_LOW 0.0f
14 #define SATURATION_HIGH 50.0f
15
16 /* Alert thresholds ( C ) */
17 #define TEMP_THRESH_HIGH 28.0f
18 #define TEMP_THRESH_LOW 25.0f
19
20 /* Median-filter window (must be odd) */
21 #define MEDIAN_WINDOW 5
22
23 /* Weighted (exponential) moving average coefficient  (0..1) */
24 #define EMA_ALPHA 0.2f
25
26 /*        Sensor catalogue        */
27 typedef enum {
28   SENSOR_DHT11 = 0, /* digital, single-wire            */
29   /* add future sensors here, e.g. SENSOR_LM35, SENSOR_NTC */
30   SENSOR_COUNT
31 } SensorId_t;
32
33 /* Alert state */
34 typedef enum { ALERT_OFF = 0, ALERT_ON = 1 } AlertState_t;
35
36 /*        Synchronisation        */
37 extern SemaphoreHandle_t xSemNewData;
38 extern SemaphoreHandle_t xSensorMutex;
39
40 void SensorData_Init(void);
41
42 /* Sensor selection */
43 void SensorData_SetActiveSensor(SensorId_t id);
44 SensorId_t SensorData_GetActiveSensor(void);
45
46 /* Raw value (straight from sensor_read) */
```

```
47 void SensorData_SetRawTemp(float val);
48 float SensorData_GetRawTemp(void);
49
50 /* After median filter */
51 void SensorData_SetFilteredTemp(float val);
52 float SensorData_GetFilteredTemp(void);
53
54 /* After weighted average (EMA) */
55 void SensorData_SetAveragedTemp(float val);
56 float SensorData_GetAveragedTemp(void);
57
58 /* Validity flag */
59 void SensorData_SetTempValid(int valid);
60 int SensorData_IsTempValid(void);
61
62 /* Alert */
63 void SensorData_SetTempAlert(AlertState_t alert);
64 AlertState_t SensorData_GetTempAlert(void);
```

Listing 14: SensorData.h - Shared Sensor Data Header

```
1  #include "SensorData.h"
2
3  SemaphoreHandle_t xSemNewData  = NULL;
4  SemaphoreHandle_t xSensorMutex = NULL;
5
6  static SensorId_t  gActiveSensor = SENSOR_DHT11;
7  static float       gRawTemp      = 20.0f;
8  static float       gFilteredTemp = 20.0f;
9  static float       gAveragedTemp = 20.0f;
10 static int         gTempValid    = 0;
11 static AlertState_t gTempAlert   = ALERT_OFF;
12
13 void SensorData_Init(void)
14 {
15     xSemNewData  = xSemaphoreCreateBinary();
16     xSensorMutex = xSemaphoreCreateMutex();
17 }
18
19 /*        Sensor selection        */
20 void SensorData_SetActiveSensor(SensorId_t id)
21 {
22     xSemaphoreTake(xSensorMutex, portMAX_DELAY);
23     gActiveSensor = id;
24     xSemaphoreGive(xSensorMutex);
25 }
```

```
26
27 SensorId_t SensorData_GetActiveSensor(void)
28 {
29     xSemaphoreTake(xSensorMutex, portMAX_DELAY);
30     SensorId_t val = gActiveSensor;
31     xSemaphoreGive(xSensorMutex);
32     return val;
33 }
34
35 /*        Raw temperature        */
36 void SensorData_SetRawTemp(float val)
37 {
38     xSemaphoreTake(xSensorMutex, portMAX_DELAY);
39     gRawTemp = val;
40     xSemaphoreGive(xSensorMutex);
41 }
42
43 float SensorData_GetRawTemp(void)
44 {
45     xSemaphoreTake(xSensorMutex, portMAX_DELAY);
46     float val = gRawTemp;
47     xSemaphoreGive(xSensorMutex);
48     return val;
49 }
50
51 /*        Filtered temperature (after median)        */
52 void SensorData_SetFilteredTemp(float val)
53 {
54     xSemaphoreTake(xSensorMutex, portMAX_DELAY);
55     gFilteredTemp = val;
56     xSemaphoreGive(xSensorMutex);
57 }
58
59 float SensorData_GetFilteredTemp(void)
60 {
61     xSemaphoreTake(xSensorMutex, portMAX_DELAY);
62     float val = gFilteredTemp;
63     xSemaphoreGive(xSensorMutex);
64     return val;
65 }
66
67 /*        Averaged temperature (after EMA)        */
68 void SensorData_SetAveragedTemp(float val)
69 {
70     xSemaphoreTake(xSensorMutex, portMAX_DELAY);
71     gAveragedTemp = val;
```

```
72      xSemaphoreGive(xSensorMutex);
73  }
74
75  float SensorData_GetAveragedTemp(void)
76  {
77      xSemaphoreTake(xSensorMutex, portMAX_DELAY);
78      float val = gAveragedTemp;
79      xSemaphoreGive(xSensorMutex);
80      return val;
81  }
82
83  /*         Validity         */
84  void SensorData_SetTempValid(int valid)
85  {
86      xSemaphoreTake(xSensorMutex, portMAX_DELAY);
87      gTempValid = valid;
88      xSemaphoreGive(xSensorMutex);
89  }
90
91  int SensorData_IsTempValid(void)
92  {
93      xSemaphoreTake(xSensorMutex, portMAX_DELAY);
94      int val = gTempValid;
95      xSemaphoreGive(xSensorMutex);
96      return val;
97  }
98
99  /*         Alert         */
100 void SensorData_SetTempAlert(AlertState_t alert)
101 {
102     xSemaphoreTake(xSensorMutex, portMAX_DELAY);
103     gTempAlert = alert;
104     xSemaphoreGive(xSensorMutex);
105 }
106
107 AlertState_t SensorData_GetTempAlert(void)
108 {
109     xSemaphoreTake(xSensorMutex, portMAX_DELAY);
110     AlertState_t val = gTempAlert;
111     xSemaphoreGive(xSensorMutex);
112     return val;
113 }
```

Listing 15: SensorData.cpp - Shared Sensor Data Implementation

A.5 SRV Layer – FreeRTOS Tasks

```
#pragma once
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>

void TaskAcquisition_Task(void *pvParameters);
```

Listing 16: TaskAcquisition.h - Acquisition Task Header

```
#include "TaskAcquisition.h"
#include "TempSensor.h"
#include "SensorData.h"

/*
 * TaskAcquisition     reads the currently selected sensor at a configurable
 * period (ACQUISITION_PERIOD_MS, 20-100 ms).  Exposes the raw value via
 * SensorData_SetRawTemp() and signals the conditioning task.
 */
void TaskAcquisition_Task(void *pvParameters)
{
    TickType_t xLastWakeTime = xTaskGetTickCount();

    for (;;) {
        SensorId_t activeSensor = SensorData_GetActiveSensor();

        float rawTemp = 0.0f;
        int   valid   = 0;

        switch (activeSensor) {
            case SENSOR_DHT11:
            default:
                rawTemp = TempSensor_Read();
                valid   = TempSensor_IsValid();
                break;
            /* Future sensors: case SENSOR_LM35: ... break; */
        }

        SensorData_SetRawTemp(rawTemp);
        SensorData_SetTempValid(valid);

        /* Signal conditioning task that fresh raw data is available */
        xSemaphoreGive(xSemNewData);
```

```
34
35          vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(
    ACQUISITION_PERIOD_MS));
36      }
37  }
```

Listing 17: TaskAcquisition.cpp - Acquisition Task Implementation

```
1  #pragma once
2  #include <freertos/FreeRTOS.h>
3  #include <freertos/task.h>
4
5  void TaskConditioning_Task(void *pvParameters);
```

Listing 18: TaskConditioning.h - Conditioning Task Header

```
1  #include "TaskConditioning.h"
2  #include "SensorData.h"
3  #include "LedDriver.h"
4
5  /*        Median filter (salt-and-pepper removal)        */
6  static float medianBuf[MEDIAN_WINDOW];
7  static uint8_t medianIdx = 0;
8  static uint8_t medianFilled = 0;
9
10 static void MedianFilter_Push(float val)
11 {
12     medianBuf[medianIdx] = val;
13     medianIdx = (medianIdx + 1) % MEDIAN_WINDOW;
14     if (medianFilled < MEDIAN_WINDOW) medianFilled++;
15 }
16
17 /* Simple insertion-sort on a small copy     return median */
18 static float MedianFilter_Get(void)
19 {
20     if (medianFilled == 0) return 0.0f;
21
22     float sorted[MEDIAN_WINDOW];
23     uint8_t n = medianFilled;
24     for (uint8_t i = 0; i < n; i++) sorted[i] = medianBuf[i];
25
26     /* Insertion sort (n     5) */
27     for (uint8_t i = 1; i < n; i++) {
28         float key = sorted[i];
29         int j = i - 1;
30         while (j >= 0 && sorted[j] > key) {
```

```
31              sorted[j + 1] = sorted[j];
32              j--;
33          }
34          sorted[j + 1] = key;
35      }
36      return sorted[n / 2];
37  }
38
39  /*        Saturation (clamp)        */
40  static float Saturate(float val)
41  {
42      if (val < SATURATION_LOW)  return SATURATION_LOW;
43      if (val > SATURATION_HIGH) return SATURATION_HIGH;
44      return val;
45  }
46
47  /*        Exponential Moving Average        */
48  static float emaValue = 20.0f;
49  static int   emaInitialised = 0;
50
51  static float EMA_Update(float newVal)
52  {
53      if (!emaInitialised) {
54          emaValue = newVal;
55          emaInitialised = 1;
56      } else {
57          emaValue = EMA_ALPHA * newVal + (1.0f - EMA_ALPHA) *
    emaValue;
58      }
59      return emaValue;
60  }
61
62  /*        Alert evaluation        */
63  static AlertState_t EvaluateAlert(float processed, AlertState_t
    current)
64  {
65      if      (processed >= TEMP_THRESH_HIGH) return ALERT_ON;
66      else if (processed <= TEMP_THRESH_LOW)  return ALERT_OFF;
67      return current;  /* hysteresis band     keep previous state */
68  }
69
70  /*
71   * TaskConditioning     signal-conditioning pipeline:
72   *    1. Saturation (clamp to physical range)
73   *    2. Median filter (salt-and-pepper / outlier removal)
74   *    3. Weighted average (EMA smoothing)
```

```
 *   4. Threshold alert evaluation
 */
void TaskConditioning_Task(void *pvParameters)
{
    for (;;) {
        /* Block until Acquisition signals new data */
        if (xSemaphoreTake(xSemNewData, pdMS_TO_TICKS(200)) != pdTRUE)
            continue;

        if (!SensorData_IsTempValid())
            continue;

        float raw = SensorData_GetRawTemp();

        /* 1. Saturation */
        float saturated = Saturate(raw);

        /* 2. Median filter (removes salt-and-pepper noise) */
        MedianFilter_Push(saturated);
        float filtered = MedianFilter_Get();
        SensorData_SetFilteredTemp(filtered);

        /* 3. Weighted average (EMA) */
        float averaged = EMA_Update(filtered);
        SensorData_SetAveragedTemp(averaged);

        /* 4. Alert evaluation on the median-filtered value (not EMA,
         *    which lags too much and can keep the alert stuck in the
         *    hysteresis band) */
        AlertState_t currentAlert = SensorData_GetTempAlert();
        AlertState_t newAlert = EvaluateAlert(filtered, currentAlert);
        if (newAlert != currentAlert) {
            SensorData_SetTempAlert(newAlert);
        }

        /* LED feedback: Green = OK, Red = alert */
        if (newAlert == ALERT_ON) {
            LedDriver_GreenOff();
            LedDriver_RedOn();
        } else {
            LedDriver_RedOff();
            LedDriver_GreenOn();
```

```
        }
    }
}
```

Listing 19: TaskConditioning.cpp - Conditioning Task Implementation

```
#pragma once
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>

void TaskReporter_Task(void *pvParameters);
```

Listing 20: TaskReporter.h - Reporter Task Header

```
#include "TaskReporter.h"
#include "LcdDisplay.h"
#include "OledDisplay.h"
#include "SensorData.h"
#include <Arduino.h>
#include <stdio.h>

static const char *SensorName(SensorId_t id) {
  switch (id) {
  case SENSOR_DHT11:
    return "DHT11";
  default:
    return "UNKNOWN";
  }
}

/*
 * TaskReporter     displays processed values, intermediate stages and
 * alerts on the OLED, LCD and via Serial (printf) at REPORTER_PERIOD_MS.
 */
void TaskReporter_Task(void *pvParameters) {
  TickType_t xLastWakeTime = xTaskGetTickCount();
  uint32_t reportCounter = 0;

  for (;;) {
    float raw = SensorData_GetRawTemp();
    float filtered = SensorData_GetFilteredTemp();
    float averaged = SensorData_GetAveragedTemp();
    int tempOk = SensorData_IsTempValid();
    AlertState_t tAlert = SensorData_GetTempAlert();
```

```
31      SensorId_t sensor = SensorData_GetActiveSensor();
32
33      /* ===== OLED 128x64 (8 lines x 21 chars at size 1) ===== */
34      char buf[22];
35      OledDisplay_Clear();
36      OledDisplay_SetTextSize(1);
37
38      OledDisplay_PrintAt(10, 0, "-- Temp Monitor --");
39
40      snprintf(buf, sizeof(buf), "Sensor: %s", SensorName(sensor));
41      OledDisplay_PrintAt(0, 10, buf);
42
43      if (tempOk) {
44        snprintf(buf, sizeof(buf), "Raw:    %.2f C", raw);
45        OledDisplay_PrintAt(0, 20, buf);
46        snprintf(buf, sizeof(buf), "Median: %.2f C", filtered);
47        OledDisplay_PrintAt(0, 30, buf);
48        snprintf(buf, sizeof(buf), "EMA:    %.2f C", averaged);
49        OledDisplay_PrintAt(0, 40, buf);
50      } else {
51        OledDisplay_PrintAt(0, 20, "Sensor ERROR");
52      }
53
54      snprintf(buf, sizeof(buf), "Alert:%s",
55               (tAlert == ALERT_ON) ? "ON " : "OFF");
56      OledDisplay_PrintAt(0, 52, buf);
57
58      snprintf(buf, sizeof(buf), "Hi:%.0f Lo:%.0f", TEMP_THRESH_HIGH,
59               TEMP_THRESH_LOW);
60      OledDisplay_PrintAt(62, 52, buf);
61
62      OledDisplay_Update();
63
64      /* ===== LCD 16x2 (backup) ===== */
65      char line1[17], line2[17];
66      if (tempOk) {
67        snprintf(line1, sizeof(line1), "T:%.1f F:%.1f   ", averaged,
    filtered);
68      } else {
69        snprintf(line1, sizeof(line1), "Sensor ERROR    ");
70      }
71      snprintf(line2, sizeof(line2), "%-7s A:%-3s   ", SensorName(
    sensor),
72               (tAlert == ALERT_ON) ? "ON" : "OFF");
73
74      LcdDisplay_PrintAt(0, 0, line1);
```

```
75      LcdDisplay_PrintAt(0, 1, line2);
76
77      /* ===== Serial Plotter (>name:value format) ===== */
78      Serial.print(">");
79      Serial.print("raw:");
80      Serial.print(raw, 2);
81      Serial.print(",median:");
82      Serial.print(filtered, 2);
83      Serial.print(",ema:");
84      Serial.print(averaged, 2);
85      Serial.print(",thresh_high:");
86      Serial.print(TEMP_THRESH_HIGH, 1);
87      Serial.print(",thresh_low:");
88      Serial.print(TEMP_THRESH_LOW, 1);
89      Serial.println(); /* \r\n */
90
91      /* ===== Structured report (every FULL_REPORT_EVERY cycles)
        ===== */
92      if ((reportCounter++ % FULL_REPORT_EVERY) == 0) {
93        printf("\n===== SENSOR REPORT (t=%lu ms) =====\n",
94               (unsigned long)(xTaskGetTickCount() *
        portTICK_PERIOD_MS));
95        printf("Active sensor : %s\n", SensorName(sensor));
96        printf("Validity      : %s\n", tempOk ? "OK" : "ERROR");
97        printf("Raw           : %.2f C\n", raw);
98        printf("Saturated+Med : %.2f C  (median window=%d)\n",
        filtered,
99               MEDIAN_WINDOW);
100       printf("EMA averaged  : %.2f C  (alpha=%.2f)\n", averaged,
        EMA_ALPHA);
101       printf("Alert         : %s  (thresholds: ON>=%.1f OFF<=%.1f)\n
        ",
102              (tAlert == ALERT_ON) ? "ON" : "OFF", TEMP_THRESH_HIGH,
103              TEMP_THRESH_LOW);
104       printf("====================================\n\n");
105     }
106
107     vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(REPORTER_PERIOD_MS
        ));
108   }
109 }
```

Listing 21: TaskReporter.cpp - Reporter Task Implementation